

Kanban Task Manager — Concepts & Mistakes Log

A running record of the CSS/Tailwind, DOM, and TypeScript concepts covered while building the project.

1. Tailwind & CSS Layout

Container padding (Bootstrap vs Tailwind)

Concept / Fix: Tailwind's **container** class only sets max-width — it has no built-in side padding like Bootstrap. Add it manually with **mx-auto px-4**, or configure default container padding once in `tailwind.config.js` (`theme.container.padding`) so every **container** use gets consistent side spacing automatically.

Mobile-first breakpoints

Mistake: Wrote the fixed/desktop size unprefixed and the flexible size behind sm:, e.g. w-45 sm:w-full — this made the field narrow on phones and full width only on larger screens (backwards).

Concept / Fix: Tailwind breakpoints (sm, md, lg...) are **minimum-width** triggers — 'apply from this size and up,' not 'apply at this size.' The unprefixed class is the base style that applies to **all** screens, most importantly small phones. Correct mobile-first order: base class = mobile layout, prefixed classes = overrides for bigger screens (w-full sm:w-45, not w-45 sm:w-full).

inline-block baseline shift

Mistake: Two sibling divs placed inline-block sat at different heights once one of them gained extra content (an error message span) below its input.

Concept / Fix: inline-block elements align to each other by their **baseline** (bottom of their content) by default, not their top edge. Extra content in one sibling shifts that baseline down, visually pushing its neighbour. Fix: align-top on both elements, or better, wrap them in **flex items-start** so they anchor to the top regardless of content height.

hidden → inline-block causing layout shift on hover

Mistake: Buttons used the pattern hidden ... group-hover/btn:inline-block to reveal edit/delete icons on hover. The card grew taller every time it was hovered.

Concept / Fix: `display:none` (hidden) removes an element from layout entirely — it reserves zero space. Switching straight to inline-block on hover suddenly adds real height/width that wasn't reserved before, pushing the card taller. Fix: keep the element in layout at all times using `opacity-0` → `group-hover:opacity-100` (optionally with `transition-opacity`) — space is always reserved, so nothing shifts.

2. Form Inputs & Validation

Textarea placeholder not showing

Mistake: Placeholder text was typed as literal content between the tags, with leading whitespace from HTML indentation. The placeholder attribute was also added, but never appeared — even on page load before typing.

Concept / Fix: A textarea's placeholder only shows when the field is **completely empty**. Any content between the opening and closing tags — including a stray newline/indentation — counts as real content and suppresses the

placeholder. Fix: use the placeholder attribute only, and close the tag immediately with no whitespace: `placeholder="..."></textarea>` with nothing in between.

:invalid pseudo-class basics

Mistake: Added `invalid:border-red-200` expecting a red border to appear only after submitting the form; it showed up immediately on page load instead.

Concept / Fix: `:invalid` / Tailwind's `invalid:` variant applies the instant an input fails its constraint (e.g. `required + empty`) — it does not wait for user interaction or a submit attempt, unlike native browser validation bubbles. Also, `:invalid` only fires at all if the input has an actual constraint (`required`, `pattern`, `minlength`, `type="email"`, etc.); a plain input with no constraints is always considered valid.

'Only show error after submit' needs JavaScript

Concept / Fix: CSS alone can't detect 'a submit was attempted.' The reliable approach: add `novalidate` to the `<form>`, handle the submit event in JS, check `form.checkValidity()` / individual field values, and toggle error classes / unhide the message span manually — rather than relying purely on CSS pseudo-classes like `:invalid` or `peer-invalid`.

:invalid only works on real form controls

Mistake: Tried adding an `invalid:` class directly to a `<span>` error message, expecting it to react on its own.

Concept / Fix: `:invalid` is a pseudo-class for actual form controls (`input`, `select`, `textarea`) only — a `span` has no validity state. To make a sibling element react to another element's state, use Tailwind's `peer` pattern: mark the input as `peer`, then style the sibling with `peer-invalid:`, `peer-focus:`, etc.

3. DOM Timing & Script Structure

Querying elements before they exist in the DOM

Mistake: All modal-related elements (`taskName`, `cancel-btn`, `delete-btn`, etc.) were queried with `getElementById` at the very top of the script — but that HTML was only injected later, inside a button's click handler. Every listener on those elements silently did nothing.

Concept / Fix: `document.getElementById(...)` only finds elements that exist in the DOM at the moment it runs. If the HTML is injected dynamically (e.g. via `innerHTML` inside a click handler), any query for elements inside it must happen **after** that injection — not at the top of the file. Fix: move those `getElementById` calls into a setup function called right after injecting the HTML.

Optional chaining on the left side of an assignment

Mistake: Wrote `document.getElementById("modal")?.innerHTML = modalDiv;` — TypeScript rejected it (ts 2779).

Concept / Fix: Optional chaining (`?.`) can't be combined with assignment — 'assign only if not null' isn't valid syntax that way. Use an explicit null check (`if (modal) { modal.innerHTML = ... }`) for safety, or the non-null assertion (`element!.innerHTML = ...`) only when certain the element always exists.

```
const modal = document.getElementById("modal");
if (modal) {
  modal.innerHTML = modalDiv;
}
```

localStorage.getItem narrowing

Mistake: Checked if (localStorage.getItem(key) != null) then called localStorage.getItem(key) again inside the block — TypeScript still complained the value could be null (ts 2345).

Concept / Fix: TypeScript narrows types per **variable**, not per repeated function call — two separate calls to getItem, even with identical arguments, are treated as two independent unknowns. Fix: store the result in a variable once, check that variable, then use the same variable inside the block.

```
const stored = localStorage.getItem("key");
if (stored != null) {
  arr = JSON.parse(stored);
}
```

Passing whole objects into inline onclick strings

Mistake: Used onclick="fn({index}, \${taskObject})" inside a template literal to hand a task object to a handler.

Concept / Fix: Interpolating an object into a string converts it to the literal text "[object Object]" — which is not valid JavaScript when placed inside an onclick attribute, causing a syntax error the moment the button is clicked. Fix: only pass simple values (numbers/short strings) through inline handlers, and look up the full object inside the function using an index into the source array.

ES module scope breaks inline onclick handlers

Mistake: After adding export {} to the top of the TypeScript file (to keep its own scope, separate from other scripts), every onclick="myFunction()" in generated HTML started throwing 'myFunction is not defined.'

Concept / Fix: export {} turns a file into an ES module, and module-level functions are **not** attached to the global window object the way plain-script functions are. Inline onclick="..." attributes only look up functions on window, so they can no longer find them. Fix: stop using onclick strings altogether — attach behaviour with addEventListener in JS instead, using data-* attributes on the elements to pass any needed info (index, action type, source list) and event delegation (closest(...)) so it keeps working after re-renders.

Duplicate event listeners after re-rendering

Mistake: A document.addEventListener('click', ...) block was placed inside each show...Tasks() render function. Since those functions run on every task move, listeners kept stacking — one click eventually triggered the same move logic multiple times, corrupting the arrays with undefined entries and crashing later renders.

Concept / Fix: Event delegation via addEventListener on document only needs to be registered **once**, since it listens for clicks bubbling up from any current or future button — it doesn't need to be reattached after innerHTML changes. Fix: register it a single time near the top of the script (after the first render), and never place it inside a function that re-runs on every render. Adding a defensive 'if (!card) return;' guard in the move/update/delete functions is good practice regardless, to prevent bad data even if a click fires unexpectedly.

Storing DOM elements instead of their values

Mistake: A task object stored the actual input elements (name: taskNameElement) instead of their text (name: taskNameElement.value). JSON.stringify on the saved tasks produced nested, mostly-empty objects in localStorage.

Concept / Fix: A variable holding an input element (e.g. taskName) refers to the whole <input> DOM node — methods, attributes, and all. .value is the one property holding what the user actually typed. DOM elements don't

serialize meaningfully with `JSON.stringify`, since their real state lives behind internal getters, not plain enumerable properties. Fix: always store `.value` (or `.value.trim()`, etc.) in the data model, and keep the full element reference only inside functions that need to read from it live.

Case-sensitive string comparisons

Mistake: Priority badge colours never applied — the code compared against "High" / "Medium" / "Low", while the `<select>` options actually stored lowercase values ("high", "medium", "low").

Concept / Fix: String comparisons in JavaScript/TypeScript are case-sensitive; `"low" !== "Low"`. Fix: compare against the same casing the data actually holds, and apply `.toUpperCase()` only at the point of *display*, not in the stored value or the comparison.

4. General Debugging Takeaways

- MIME type errors ("Refused to apply style... not a supported stylesheet MIME type") usually mean the requested file path doesn't actually exist, so the dev server returns an HTML error page instead of the real file — check the exact folder name, casing, and try opening the asset URL directly in the browser to confirm.
- "Cannot set properties of null (setting 'innerHTML')" always means `getElementById` returned null — the id is missing, misspelled, or the element doesn't exist yet at that point in execution.
- Colours that don't visually work are often a plain contrast issue (e.g. a white icon on a white background) before they're a class-name or specificity bug — check computed contrast first.
- When several buttons need both an index and extra context (which array, what action), pass that context via `data-*` attributes rather than encoding it into the function name or inline handler string.